UNIVERSIDAD NACIONAL DE SAN ANTONIO ABAD DEL CUSCO

FACULTAD DE INGENIERÍA ELÉCTRICA, ELECTRÓNICA, INFORMÁTICA Y MECÁNICA

ESCUELA PROFESIONAL DE INGENIERÍA INFORMÁTICA Y DE SISTEMAS



INFORME DE LABORATORIO N° 7

Sprint Planning y Ejecución Inicial

Proyecto: Centro de Capacitación e Inducción – CGB Academy

Sprint 2: Onboarding (09/11/2026 – 23/11/2026)

Datos de la Asignatura y Repositorios Oficiales

Curso: Ingeniería de Software I (IF614)
Docente: Ing. Lisha Sabah Diaz Caceres
Semestre Académico: 2026-I
Repositorio en GitHub: https://github.com/milith0kun/proyecto-ing-software
Backlog en Jira Cloud: https://cgb-academy.atlassian.net (Proyecto: **CGB**)

Integrantes del Grupo de Trabajo:

N°	Apellidos y Nombres	Código	Rol en el Sprint 2
1	Barazorda Cuellar, Hector	145003	Desarrollador
2	Carpio Hermoza, Alex	164563	Desarrollador
3	Quispe Mamani, Domingo de Guzman	191978	Desarrollador
4	Saire Bustamante, Edmil Jampier	174449	Product Owner y Scrum Master
5	Ticona Jancco, Ronaldo	211816	Desarrollador

Cusco, Perú – Noviembre de 2026

Índice

1. Objetivo del Laboratorio **2**

2. Fundamento Teórico **2**

3. Herramientas Utilizadas **2**

4. Desarrollo de las Actividades **2**
4.1. Parte 1: Definición del Sprint 2
4.2. Parte 2: Selección de Historias y Capacidad 3
4.3. Parte 3: Descomposición en Tareas 4
4.4. Parte 4: Inicio del Desarrollo 5

5. Entregables **7**

6. Trabajo Extra: Avance de Implementación (CGB-31 / HU-007) **7**

7. Conclusiones **8**

8. Referencias Bibliográficas **8**

1. Objetivo del Laboratorio

Planificar el segundo sprint del proyecto CGB Academy: definir su duración y objetivo, seleccionar las historias de usuario según la capacidad del equipo, descomponerlas en tareas, asignarlas y dejar evidencia del inicio del desarrollo (ramas en Git y primer avance con TDD). Los datos de Jira citados se leyeron el 09/11/2026 mediante la API del proyecto **CGB**.

2. Fundamento Teórico

- **Scrum y Sprint Planning:** el equipo acuerda qué se entregará en el sprint (*objetivo*), por qué es valioso y cómo se construirá. La selección parte de la capacidad real del equipo y no del deseo de entregar más (Schwaber y Sutherland, 2020).
- **Sprint Backlog:** conjunto de historias seleccionadas más el plan para entregarlas. En Jira se materializa como las historias asignadas al sprint con sus subtareas.
- **Definition of Done (DoD):** criterios comunes y verificables para considerar terminada una historia; en el proyecto están en `docs/definition-of-done.md`.
- **Estrategia de ramas:** *GitHub Flow*. Una sola rama permanente (`main`), una rama `feature` corta por historia, Pull Request revisado por otra persona y *Squash and merge*.

3. Herramientas Utilizadas

- **Jira Software Cloud:** proyecto CGB, sprint 2 (id 2), historias y subtareas.
- **GitHub:** repositorio `milith0kun/proyecto-ing-software`, ramas y Pull Requests.
- **Node.js, `tsx` y Prisma 6.19:** pruebas automáticas y validación del esquema de datos.

4. Desarrollo de las Actividades

4.1. Parte 1: Definición del Sprint

Elemento	Definición
Sprint	Sprint 2: Onboarding (estado *activo* en Jira)
Duración	2 semanas: 09/11/2026 – 23/11/2026
Product Owner y Scrum Master	Edmil Jampier Saire Bustamante
Desarrolladores (4)	Alex Carpio, Domingo Quispe, Ronaldo Ticona y Héctor Barazorda

Tabla 1: Datos del Sprint 2.

Objetivo del Sprint 2 (cargado en Jira)

Permitir que el administrador defina áreas, puestos y rutas de onboarding y registre colaboradores; que cada colaborador active su cuenta, consulte *Mi Onboarding* y registre su avance; y dejar configurado y disponible el microtest formativo.

Alcance excluido: constancias, insignias y certificados (fuera del MVP, ver CGB-51). El reporte de avance (CGB-71, HU-015) permanece en el Sprint 3.

4.2. Parte 2: Selección de Historias y Capacidad

Cambio de alcance respecto al Laboratorio 5. Las historias CGB-61 (HU-013) y CGB-66 (HU-014), con 10 SP, se adelantaron del Sprint 3 al Sprint 2. El sprint pasó de 22 a **32 SP**, y el Sprint 3 queda con CGB-71 (5 SP). Esta decisión de alcance requiere aprobación del Product Owner registrada en Jira.

Capacidad. En el Sprint 1 se entregaron 19 SP con 3 desarrolladores (≈ 6,3 SP por persona); con 4 desarrolladores la capacidad equivalente es ≈ 25 SP. El Sprint 2 compromete 32 SP, es decir 8 SP por persona, un 28 % por encima de esa capacidad. Es un compromiso **ambicioso**: si el sprint se retrasa, las candidatas a salir son HU-014 (CGB-66) y HU-012 (CGB-56).

Clave	Historia	SP	Responsable (Jira)	Depende de
CGB-31	HU-007: Crear área y puestos de trabajo	3	Alex Carpio	–
CGB-36	HU-008: Crear y secuenciar ruta de onboarding	5	Domingo Quispe	CGB-31
CGB-41	HU-009: Registrar colaboradores y asignar rutas	3	Ronaldo Ticona	CGB-31, 36
CGB-46	HU-010: Activar cuenta mediante token	3	Domingo Quispe	CGB-41
CGB-51	HU-011: Consultar panel personal Mi Onboarding	5	Ronaldo Ticona	CGB-36, 46
CGB-56	HU-012: Completar capacitación y registrar avance	3	Héctor Barazorda	CGB-51
CGB-61	HU-013: Configurar microtest formativo	5	Alex Carpio	HU-003
CGB-66	HU-014: Realizar microtest formativo interactivo	5	Héctor Barazorda	CGB-61
	Total	**32**	8 SP por persona	

Tabla 2: Sprint Backlog del Sprint 2 con los responsables vigentes en Jira.

Antes del reparto, Jira concentraba 15 SP en Domingo, 11 en Alex, 6 en Ronaldo y ninguno en Héctor. Se reasignaron CGB-46, CGB-51, CGB-56 y CGB-66 junto con sus 16 subtareas para que cada integrante tenga una historia de 3 SP y una de 5 SP.

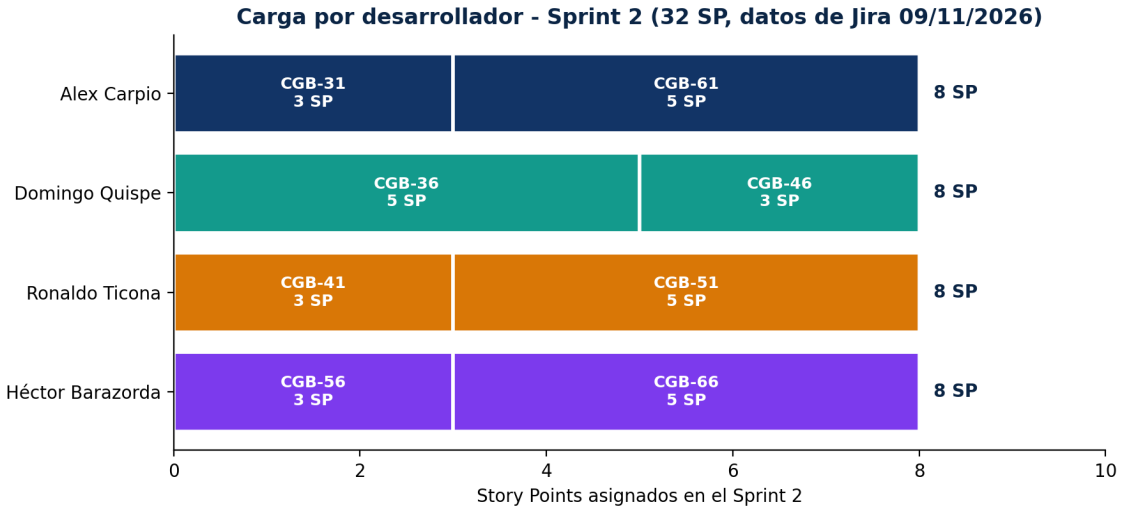


Figura 1: Carga por desarrollador en el Sprint 2, generada con datos leídos de Jira el 09/11/2026.

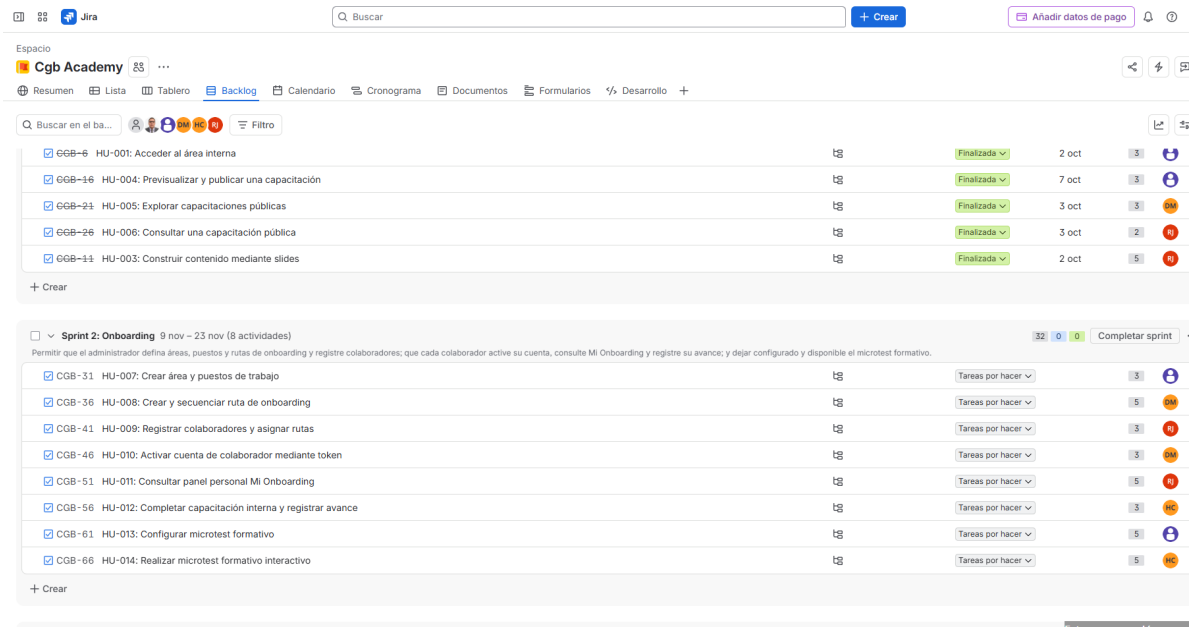


Figura 2: Backlog en Jira Cloud: Sprint 2 con su objetivo, 8 actividades, 32 SP y el responsable de cada historia (avatares). Captura del equipo.

Riesgos identificados.

1. Carga de 32 SP por encima de la capacidad equivalente.
2. Dependencias en cadena (CGB-31 → 36 → 41 → 46 → 51 → 56 y CGB-61 → 66). Mitigación: acordar el esquema Prisma el primer día.
3. CGB-51 y CGB-56 llevan la etiqueta `evidence-pending`: aún no se entrevistó ni observó al colaborador y no se inventan hallazgos.
4. Doble rol de Edmil (Product Owner y Scrum Master); las revisiones de código las hacen los desarrolladores entre sí.
5. Privacidad (AG-C01): el avance de cada colaborador es dato personal y solo debe verlo el propio colaborador y el administrador.

Límite WIP. Con cuatro desarrolladores, el límite de la columna *Desarrollo* sube de 3 a **4** (una historia activa por desarrollador); *Revisión* se mantiene en 2. El cambio está en `docs/FLUJO_DE_TRABAJO.md`, en el README y en el informe del proyecto.

4.3. Parte 3: Descomposición en Tareas

Cada historia tiene en Jira cuatro subtareas del ciclo TDD, asignadas al mismo responsable que la historia y todas en *Tareas por hacer*:

Historia	Subtareas: T1 RED / T2 GREEN / T3 REFACTOR / T4 VALIDACIÓN
CGB-31	CGB-32 / 33 / 34 / 35
CGB-36	CGB-37 / 38 / 39 / 40
CGB-41	CGB-42 / 43 / 44 / 45
CGB-46	CGB-47 / 48 / 49 / 50
CGB-51	CGB-52 / 53 / 54 / 55
CGB-56	CGB-57 / 58 / 59 / 60
CGB-61	CGB-62 / 63 / 64 / 65
CGB-66	CGB-67 / 68 / 69 / 70

Tabla 3: Subtareas existentes en Jira. Sus descripciones están vacías; el desglose siguiente es una propuesta para completarlas.

Contrato de datos del primer día (extiende `prisma/schema.prisma`): `Area`, `PuestoTrabajo`, `RutaOnboarding` y `PasoRuta`, token de activación, `Avance` y las entidades del microtest.

Historia	Desglose técnico propuesto
CGB-31	Modelos de área y puesto; validación con nombre único; API protegida por rol administrador; vista en acordeón.
CGB-36	Modelos de ruta y pasos; solo capacitaciones internas publicadas; API de creación y reordenamiento.
CGB-41	Alta de colaborador inactivo con puesto y ruta; correo duplicado; formulario de alta.
CGB-46	Token de un solo uso con expiración; casos inválido, expirado y reutilizado; activación de cuenta.
CGB-51	Consulta de la ruta del colaborador y porcentaje de avance; aislamiento entre usuarios.
CGB-56	Modelo de avance; registro idempotente; integración con el visor de slides.
CGB-61	Microtest con preguntas y opciones; validación de configuración; pantalla de administración.
CGB-66	Intento, calificación y retroalimentación formativa; pantalla interactiva.

Tabla 4: Desglose técnico propuesto por historia.

4.4. Parte 4: Inicio del Desarrollo

Estrategia de ramificación elegida: GitHub Flow. Se descartó Git Flow (`develop`, `release`) porque el equipo es pequeño, los sprints duran dos semanas y existe una sola versión activa. La regla está en `docs/FLUJO_DE_TRABAJO.md`.

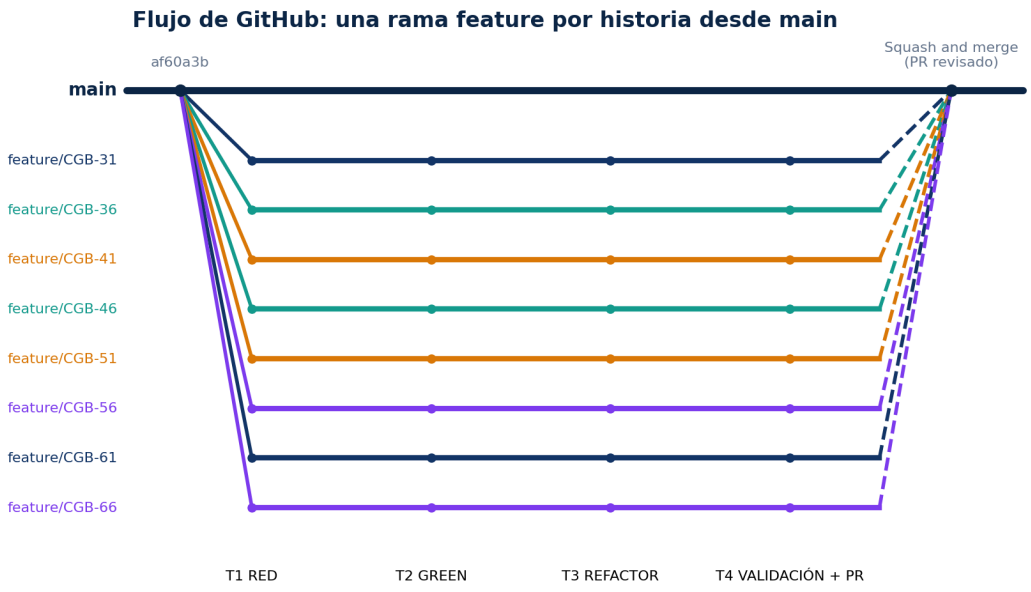


Figura 3: Ramas del Sprint 2 creadas desde `main` (commit af60a3b); cada una cierra con un Pull Request revisado.

Ramas creadas y publicadas en GitHub (formato `tipo/clave-jira-resumen`):

Rama	Historia	Responsable
`feature/CGB-31-areas-y-puestos`	CGB-31	Alex Carpio
`feature/CGB-36-ruta-onboarding`	CGB-36	Domingo Quispe
`feature/CGB-41-registrar-colaboradores`	CGB-41	Ronaldo Ticona
`feature/CGB-46-activar-cuenta-token`	CGB-46	Domingo Quispe
`feature/CGB-51-mi-onboarding`	CGB-51	Ronaldo Ticona
`feature/CGB-56-completar-capacitacion`	CGB-56	Héctor Barazorda
`feature/CGB-61-configurar-microtest`	CGB-61	Alex Carpio
`feature/CGB-66-realizar-microtest`	CGB-66	Héctor Barazorda

Tabla 5: Ramas del Sprint 2. Todas parten de `origin/main`; solo CGB-31 tiene ya un commit propio.

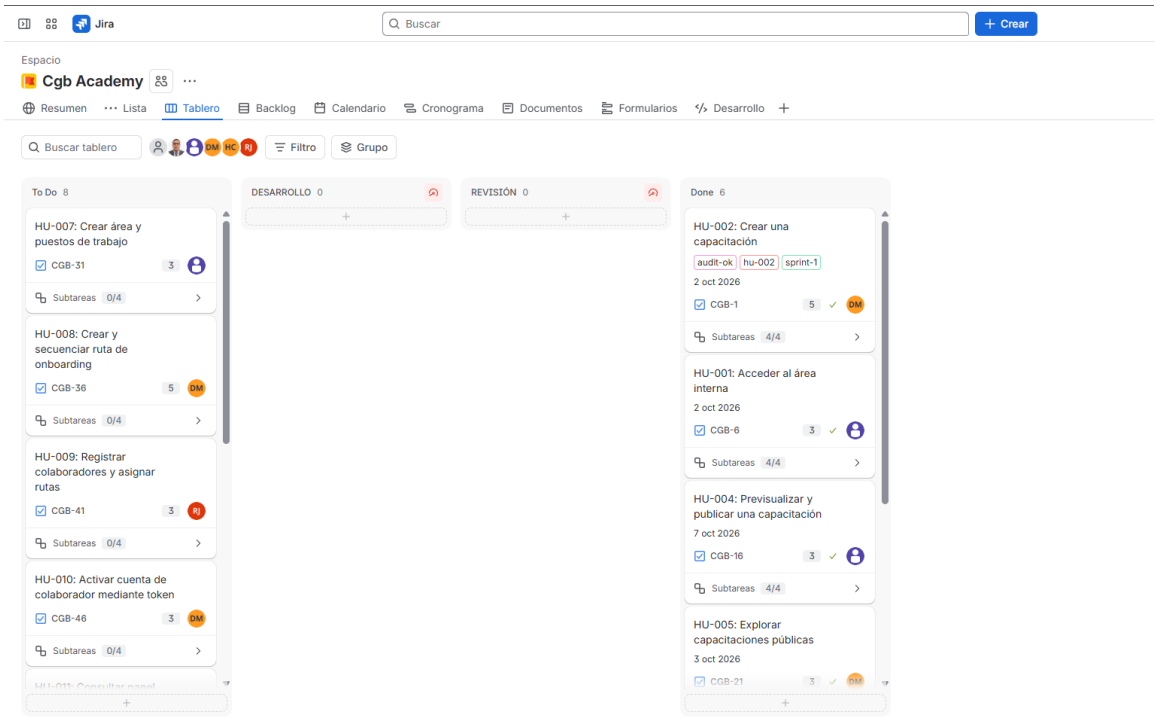


Figura 4: Tablero en Jira Cloud al inicio del Sprint 2: 8 historias en *To Do* con 0/4 subtareas, columnas *Desarrollo* y *Revisión* vacías y las historias del Sprint 1 en *Done*. Captura del equipo.

Estado en Jira. Responsables, objetivo del sprint y subtareas están cargados; se creó la tarea CGB-78 para la alineación documental; CGB-76 y CGB-77 se asignaron a Edmil; se enlazaron en Jira los Pull Requests y commits de las historias ya integradas (CGB-1, 6, 11, 16 y 21). Como muestra el tablero, todavía no se movió ninguna historia a *En desarrollo* (0 en Desarrollo y 0 en Revisión).

5. Entregables

Entregable	Estado
Sprint Backlog definido (8 historias, 32 SP, objetivo cargado en Jira)	Cumplido
Tareas asignadas (historias y 32 subtareas con responsable)	Cumplido
Ramas Git creadas y publicadas (8)	Cumplido
Primer commit de desarrollo (CGB-31)	Cumplido
Historias movidas a *En desarrollo* en Jira	Pendiente
Límite WIP de 4 configurado en el tablero de Jira	Pendiente
Capturas del backlog y del tablero del Sprint 2 con el reparto actual	Cumplido
Captura de las ramas en GitHub y del commit de CGB-31	Pendiente
Aprobación y fusión del PR #7 (documentos de flujo, DoD y configuración)	Pendiente de revisión

Tabla 6: Estado de los entregables del laboratorio.

6. Trabajo Extra: Avance de Implementación (CGB-31 / HU-007)

Se inició la implementación de la primera historia de la cadena de dependencias en `feature/CGB-31-areas-y-` commit `42eb4a4`, siguiendo TDD:

1. **RED (T1):** `tests/organizacion.test.mjs` con 7 pruebas que cubren CA-01 a CA-04 (alta de área, puestos vinculados al área, duplicados y jerarquía con conteo de colaboradores).

Falló porque el módulo aún no existía.

2. **GREEN (núcleo de T2):** `src/lib/organizacion.ts` con validación, detección de duplicados sin distinguir mayúsculas ni tildes y armado de la jerarquía; y los modelos `Area` y `PuestoTrabajo` en `prisma/schema.prisma`.

Comprobaciones ejecutadas: `npx tsx -test tests/organizacion.test.mjs` → 8 de 8 pruebas aprobadas; `prisma validate` (Prisma 6.19.3) → esquema válido.

Limitaciones declaradas

`npm run lint` no se ejecutó porque el equipo local no tiene `node_modules`. En la suite completa fallan 4 archivos de prueba (`autenticacion`, `publicacion-integracion`, `visor-capacitacion` y `vista-previa`) por dependencias no instaladas (`bcryptjs`, `next`, `happy-dom`); se repetirá tras `npm ci`. Faltan de CGB-31 los endpoints de `/api/organizacion`, la vista en acordeón, las pruebas de integración y el Pull Request.

7. Conclusiones

1. La capacidad del equipo debe medirse con la velocidad real: al sumar un cuarto desarrollador y adelantar dos historias, el sprint pasó a 32 SP, por encima de los ≈ 25 SP que sugiere el Sprint 1. Se dejó acordado qué historias salen primero si hay retraso.

2. Repartir la carga explícitamente (8 SP por persona) evitó que un integrante concentrara 15 SP mientras otro no tenía trabajo, y se reflejó en Jira para que el tablero sea la fuente de verdad.

3. Una sola rama permanente con ramas cortas por historia simplifica la integración para un equipo de cuatro personas, siempre que cada Pull Request lo revise alguien distinto del autor.

4. El laboratorio queda con pendientes que dependen de personas, no del documento: la aprobación del PR #7, el movimiento de historias a *En desarrollo* y las capturas de GitHub.

8. Referencias Bibliográficas

- Schwaber, K. y Sutherland, J. (2020). *The Scrum Guide*. https://scrumguides.org/scrum-guide.html
- GitHub Docs. *GitHub flow*. https://docs.github.com/
- Conventional Commits 1.0.0. https://www.conventionalcommits.org/en/v1.0.0/
- Cohn, M. (2005). *Agile Estimating and Planning*. Upper Saddle River, NJ: Prentice Hall PTR.